

📖 What is Recursion?

Recursion is a programming technique where a function solves a problem by calling a copy of itself to work on a smaller, slightly simpler version of the original problem.

Key Properties of a Recursive Function

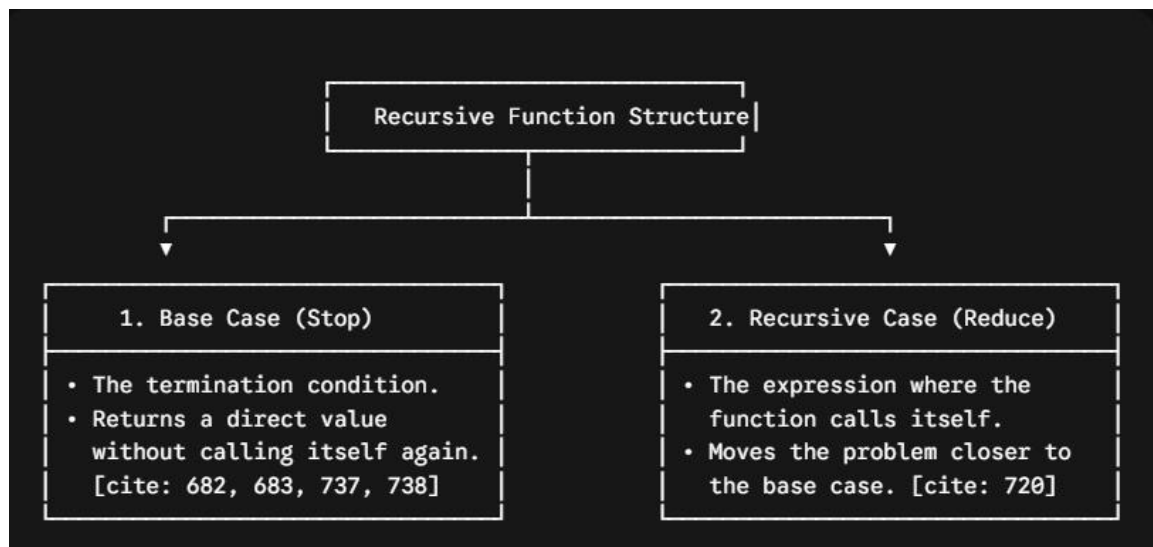
Self-Invocation: The function contains an internal statement that calls itself.

Problem Reduction: Each progressive self-call passes arguments that break the larger problem down into a smaller, more digestible sub-problem.

The Stack Frame Lifecycle: Every recursive call pauses the current execution state and pushes a new frame onto the system's call stack memory. These frames remain open until a resolution condition is met.

🔗 The Core Anatomy of a Recursive Solution

To prevent a recursive function from running indefinitely and causing a system crash (**Stack Overflow**), it must be built with two mandatory logical cases:



🔧 How to Approach a Recursive Problem

When trying to solve a problem using recursion, follow these three engineering steps:

Assume the Function Exists: Believe that your function is already successfully created and knows how to compute smaller subsets of your target problem.

Formulate the Recursive Case: Define the mathematical or logical relationship between the main problem $f(n)$ and the sub-problem $f(n-1)$.

Establish the Base Case: Determine the absolute smallest input where the answer is known instantly without any further math.

Code Analysis 1: The Summation Tree

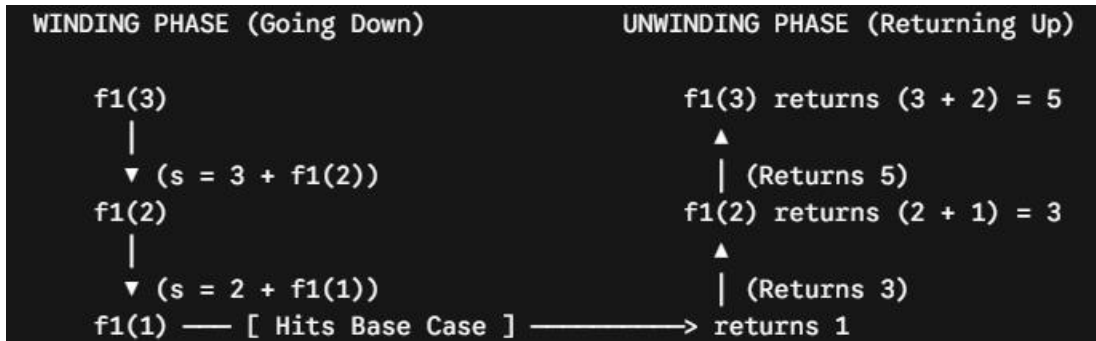
Let's look at the recursive calculation of a number's structural summation where $f(n) = n + f(n-1)$.

Implementation Blueprint

```
#include <stdio.h>
int f1(int n); ----> Prototype
int main()
{
    int k;
    k = f1(3); ----> Initial call with n = 3
    printf("%d\n", k); ----> Prints final aggregated result
    return 0;
}
int f1(int n)
{
    int s;
    if (n == 1) ----> Base Case
    {
        return 1;
    }
    s = n + f1(n - 1); ----> Recursive Case
    return s;
}
```

The Execution Tree (f1(3))

When $f1(3)$ is triggered, the execution branches out downward until it reaches the base case, and then unwinds back upward to calculate the final value:



Code Analysis 2: Calculating Factorial (n!)

The factorial of a number (n!) is defined as $n * (n-1) * (n-2) * \dots * 1$

Recursive Relationship: $\text{fact}(n) = n * \text{fact}(n-1)$

Base Case Condition: If $n == 0$, the value is mathematically predefined as 1

Implementation Blueprint

```

#include <stdio.h>
int fact(int n); ----> Function Declaration
int main()
{
    int n, s;
    printf("Enter the value of n = ");
    scanf("%d", &n);

    s = fact(n); ----> Triggers recursive solver
    printf("Factorial of given number = %d\n", s);
    return 0;
}
int fact(int n)
{
    if (n == 0) ----> Base Case
    {
        return 1; ----> Terminates further deep scaling
    }
    return (n * fact(n - 1)); ----> Recursive Case
}
    
```

Interactive Stack Trace for fact(4)

Let's watch how the computer tracks execution steps inside its system memory allocations:

Current Stack Call Frame	Operation Performed	Wait Status	Return Resolution
fact(4)	$4 * \text{fact}(3)$	Waiting on $\text{fact}(3)$	$4 * 6 = 24$ (Final Answer)
fact(3)	$3 * \text{fact}(2)$	Waiting on $\text{fact}(2)$	$3 * 2 = 6$
fact(2)	$2 * \text{fact}(1)$	Waiting on $\text{fact}(1)$	$2 * 1 = 2$
fact(1)	$1 * \text{fact}(0)$	Waiting on $\text{fact}(0)$	$1 * 1 = 1$
fact(0)	<i>Base Case Hit!</i>	None	Returns 1 directly

⚠ Important Note:

Recursion is one of the most critical topics in computer science. Please take the time to learn and understand its underlying stack mechanics thoroughly, and ensure you practice plenty of programming problems to build your confidence.

To practice and test your logic, make sure to go through the **Coding Set** provided in the same folder: **09-Recursion**.

📖 **Stuck or Prefer a Visual Walkthrough?** If you are not able to understand through these notes or you need to explore these concepts more visually, you can review my handwritten notes PDF provided in the same folder: **09-Recursion**